AIAA 4051 NLP Final project 26Fall

All projects will be assessed using the unified course evaluation format.

Project 1

Project title: Conformal Prediction for Reliable Short- and Long-Form Question Answering

Project manager: Rui Xu

Email: rxu233@connect.hkust-gz.edu.cn

Project description

Large language models can generate incorrect answers with high confidence. Conformal prediction provides a framework for calibrating uncertainty, but its application depends on what constitutes a reliable response.

This project investigates two approaches. ConU constructs sets of sampled answers intended to contain at least one correct answer. Conformal factuality decomposes a response into claims and removes uncertain claims to improve the factuality of the retained content. These objectives suggest different advantages for short answers and longer responses containing multiple claims. Students will reproduce both approaches, investigate their strengths and limitations, and explore a simple way to combine their ideas. ConU paper , Conformal factuality paper

Objectives

- Reproduce two conformal methods for language generation.
- Understand the distinction between covering a correct answer and controlling the factuality of retained claims.
- Investigate how answer length, sampling, and calibration affect performance.
- Implement and evaluate one modest extension combining ideas from the two methods.

Codebases and datasets

- **Short-form QA:** ConU repository , using a subset of TriviaQA.
- **Long-form generation:** Conformal factuality repository , using FActScore biography prompts. The repository includes annotated claims and saved outputs that can support initial reproduction. Cached outputs and annotations

One dataset per setting and one inference model are sufficient. Students may begin with cached outputs and then generate a small collection of new responses.

Tasks

Part 1: Reproduce the basic methods

- **Run the ConU pipeline:** sample answers, compute uncertainty scores, calibrate a threshold, and construct answer sets.
- **Run the conformal factuality pipeline:** decompose responses into claims, score the claims, calibrate a threshold, and filter responses.
- Reproduce one representative result from each repository on a manageable subset.
- Explain the input, output, calibration procedure, and correctness criterion of each method.

Report answer-set coverage and size for ConU. For conformal factuality, report the proportion of responses whose retained claims are all correct, the proportion of original claims retained, and the frequency of empty responses.

Part 2: Understand suitability and failure cases

Use experimental examples to investigate the following questions:

- Why might a set of alternative answers be useful for short-form QA?
- Why might claim filtering be more useful for a long response containing both correct and incorrect claims?
- What happens when repeated sampling never produces a correct answer?
- Can filtering make a response factually correct but too incomplete to answer the question?

Present several successful and unsuccessful examples. Explain why answer-set coverage and factuality are different evaluation objectives, rather than treating their numerical values as interchangeable.

Inspect whether either implementation excludes difficult examples. In particular, the ConU implementation filters out examples without an acceptable sampled answer before splitting the data. Report the excluded proportion and examine performance on the unfiltered examples as well. ConU implementation

Part 3: Conduct a controlled empirical study

- Evaluate two target levels, such as $1 - \alpha = 0.8$ and 0.9.

- **Investigate one additional factor:** the number of sampled responses, calibration-set size, or response length.
- Visualize the tradeoff between reliability and usefulness: answer-set size for short-form QA, and retained information for long-form generation.
- Keep test examples separate from calibration and any parameter tuning. Reuse generated outputs where possible to reduce computational cost.

Students should explain observed trends and unexpected results; reproducing every experiment from the original papers is unnecessary.

Part 4: Explore a simple combination

Implement one small extension that transfers an idea from one method to the other.

A suggested direction is to combine ConU's semantic comparison of sampled answers with claim filtering:

1. Generate several responses to the same question and decompose them into claims.

2. Use semantic similarity to identify equivalent claims across responses.

3. Score each claim by the proportion of responses containing an equivalent claim.

4. Use these scores in the conformal factuality filtering procedure and recalibrate its threshold.

Compare this extension with an existing claim-scoring method at the same sampling budget. Investigate whether it retains more useful claims at comparable factuality, and examine failures caused by repeated but incorrect claims.

A well-executed negative result is acceptable. Assessment should reward sound reasoning, reproducible experiments, and honest analysis rather than improvement alone. A new theoretical guarantee is not required.

Final Report and Deliverables

Submit a report covering motivation, methods, experimental setup, results, failure analysis, and conclusions, together with runnable code, configurations, plots, a poster, and individual contribution statements.

Recommended computational scope

Hardware: 1 × NVIDIA A6000.

Proposed allocation cap: 20 GPU-hours per team for the complete project. This is a planning limit, not a measured runtime estimate.

No model training is required. Use the cached generations and existing annotations for the main reproduction and calibration experiments. Fix one inference model for any new local generations.

Use one manageable subset per setting. For the supplementary generation and combination study, use at most 50 prompts per setting and up to five responses per prompt. Select these prompts before inspecting model outcomes, and reuse the same outputs across target levels and compatible comparisons.

Keep calibration, development, and test examples disjoint. A small pilot must account for generation, semantic comparisons, claim scoring, and evaluation, including any auxiliary models. New generations supplement the cached-output experiments; the small supplementary subset alone should not be used to claim reliable coverage or factuality guarantees.

Project 2

Project title: Parameter Importance Analysis of Large Language Models in Continual Learning

Project manager: Yazheng Liu

Email: yliu533@connect.hkust-gz.edu.cn

Project description

This project aims to investigate how the importance of model parameters in large language models (LLMs) changes across different tasks under both task-specific fine-tuning and continual learning settings. While existing interpretability methods, such as Layer-wise Relevance Propagation (LRP)[1] and AttnLRP[2], mainly focus on attributing model predictions to input tokens and intermediate representations, relatively limited work has explored how these attribution principles can be extended to analyze the importance of model parameters across tasks.

In this project, we will study a fully fine-tuned Llama-3.2-3B-Instruct model using three tasks from either Concept-1K or TRACE. The required study uses one benchmark only. We aim to characterize which parameters are important for individual tasks, how much the important parameter sets overlap across different tasks, and how parameter importance evolves when tasks are learned sequentially in a continual learning setting.

Tasks

Part 1: Reproduce the algorithm presented in the AttnLRP paper

Use Llama-3.2-3B-Instruct as the base model. Choose three tasks from one benchmark, either TRACE or Concept-1K, before examining experimental outcomes. Perform full-parameter fine-tuning independently on each selected task, always starting from the same base checkpoint. Use the same selected tasks in the subsequent sequential study.

For each independently fine-tuned task-specific model, apply AttnLRP to calculate the contribution of each input token at each layer of the model, visualize the resulting relevance scores, and evaluate the quality of the explanations using faithfulness metrics. Analyze whether different tasks exhibit distinct token-level relevance patterns and compare the attribution behavior across layers and tasks.

Part 2: Parameter Importance Attribution across Individual Tasks

- Based on the Attributing Non-linear Attention rule introduced in AttnLRP, redesign, extend, or adapt the relevance propagation mechanism to estimate the importance of each parameter at each layer. The objective is to assign an importance score to individual parameters or parameter groups according to their contribution to the model output.
- Using the independently fine-tuned task-specific models obtained in Part 1, compute parameter importance for each of the three selected tasks. Identify the most important parameters for each task and quantitatively measure the overlap between important parameter sets across different tasks. Possible comparison metrics may include top-k overlap ratio, Jaccard similarity, rank correlation, or other suitable measures.
- Analyze whether different tasks rely on shared or task-specific parameters, how the overlap varies across transformer layers, and whether particular modules or layers consistently contain parameters that are important across multiple tasks.

Part 3: Parameter Importance Dynamics in Sequential Continual Learning

- Perform Sequential Continual Learning (SEQ) on the same three selected tasks using Llama-3.2-3B-Instruct with full-parameter fine-tuning. Preserve their relative order in the chosen benchmark, resulting in one continually updated model with a saved checkpoint after each task. Use the same per-task training subsets and training budget as in the independent fine-tuning study.

- After each stage of continual learning, apply the parameter importance method developed in Part 2 to evaluate the importance of model parameters for both the current task and previously learned tasks. Compute the overlap between important parameter sets across tasks
- Investigate how parameter importance changes during sequential learning, including whether parameters important to earlier tasks are reused, overwritten, or replaced when new tasks are learned.

GPU requirement

Hardware: 1 × NVIDIA A6000.

Proposed allocation cap: 50 GPU-hours per team for the complete project. This is a planning limit, not a measured runtime estimate.

The required training comprises three independent task-specific runs and one three-stage sequential run. Use one seed, up to 500 training examples per task, and at most one pass through each selected training subset. Fix the subsets and configuration before the main comparison; preserve class coverage where the benchmark requires it.

Use a fixed held-out attribution subset of at most 64 examples per task. Reuse these examples across model checkpoints, and compute attribution only at the final independent checkpoints and after each sequential-learning stage. Report results for the selected tasks and subsets rather than presenting them as full-benchmark results.

The pilot must include a complete optimizer step and an attribution pass on the stated GPU. Reduced precision, gradient checkpointing, or optimizer-state offloading may be used where supported, but all model parameters must remain trainable. Adapter-only fine-tuning is not a substitute for the required full-parameter study. Confirm memory feasibility before launching the full experiment.

Optional Extension

Additional tasks or a second benchmark may be studied only after completing the required analysis and within the confirmed allocation. They are not required for a complete project.

Dataset and Code

The code for AttnLRP is in the following link:

AttnLRP implementation

The dataset is in the following link :

Continual-learning benchmark code

Related Work

[1] Bach S, Binder A, Montavon G, et al. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation[J]. PloS one, 2015, 10(7): e0130140.

[2] Achtibat R, Hatefi S M V, Dreyer M, et al. Attnlrp: attention-aware layer-wise relevance propagation for transformers. ICML, 2024.

Project 3

Project Title: Learning from Failure: Evaluating Reflection in LLM-Based Household Agents

Project manager: Rufeng Chen

Email: rchen514@connect.hkust-gz.edu.cn

Project Description

This final project investigates whether a Large Language Model (LLM) can improve its task planning by reflecting on previous failures. Students will work within the text-based ALFWorld environment, where household activities are represented through language observations and executable commands. Tasks involve finding and moving objects, or completing prerequisite operations such as cleaning an object before placing it in a specified location. This provides an accessible setting for studying embodied task planning without requiring visual perception or physical robot control.

Students will build on the existing Reflexion implementation rather than develop a new agent platform. In Reflexion, an agent generates written feedback after an unsuccessful attempt and uses that feedback during a later attempt. Here, learning refers to adapting through text memory rather than updating model parameters.

The central question is:

Does changing the content of a reflection help an agent avoid repeating its mistakes and complete a task on a subsequent attempt?

Students will reproduce existing agents, design a focused modification to the reflection prompt, and evaluate the modification through controlled experiments. The project does not require model training, a new simulator, or a complex memory system.

Part 1: Basic Reproduction

This phase familiarizes students with ALFWorld and the existing agent implementation.

1. Environment Familiarization

Run the supplied text-based ALFWorld environment and complete a short manual interaction.

Identify the task instruction, environmental observation, command format, and environment-reported outcome. Explain how an action changes the environment and how the resulting feedback becomes available to the agent.

Expected output: A short interaction record demonstrating that the environment runs correctly, accompanied by a brief explanation of the task and action interface.

Students should use the prepared environment rather than modify the simulator or create new task definitions.

2. Baseline Reproduction

Run Condition A: No Reflection and Condition B: Original Reflection on the development tasks.

Save complete attempts, including task instructions, observations, generated actions, outcomes, and any reflections. Demonstrate a successful tutorial trajectory and document unsuccessful attempts when they occur.

Expected output: Baseline results on the development tasks and complete interaction records that demonstrate the behavior of both conditions.

Students are expected to reproduce the agent workflow, not the exact success rates reported in the original paper.

3. Reflection and Memory Inspection

Trace one unsuccessful attempt through reflection generation and the next attempt.

Verify that the task restarts from its original environment state, while Condition B retains the written reflection. Explain which information is preserved and which information must be obtained again after the reset.

For example, an object picked up during the previous attempt is not automatically held by the agent after the environment resets, even though information about that object may appear in the reflection.

Expected output: An annotated example showing the failed attempt, generated reflection, and subsequent attempt. A supplied example may be used if the development runs contain no suitable failure.

Part 2: Reflection Design and Evaluation

This phase focuses on one manageable modification and a controlled comparison.

4. Student-Designed Reflection

Implement Condition C by modifying the reflection instructions.

A suggested reflection structure is:

- **Observed evidence:** What actions and environmental feedback are directly supported by the previous interaction?
- **Likely failure and remaining uncertainty:** What probably prevented completion, and what cannot yet be established?
- **Next-attempt plan:** What should the agent do differently after the task restarts?

Students may refine this structure, but the modification should remain focused on the reflection prompt. The model, action-generation prompt, existing examples, and memory mechanism must remain unchanged.

The original Reflexion prompt already requests a concise new action plan that addresses previous mistakes. Therefore, the student contribution is a specific change in how that plan is generated, rather than introducing reflection or planning for the first time.

Develop and revise the prompt using development tasks only. Do not add a new model, planner, retrieval system, or source of task information.

Expected output: The final reflection prompt, a brief explanation of its intended effect, and examples generated during development.

5. Controlled Comparison

Evaluate all three conditions using the shared experimental protocol below.

Compare task success, recovery after initial failure, and model-use cost. Every condition must receive the same maximum number of attempts and the same interaction limit.

Discuss whether the observed results support the intended benefit of the revised prompt. Include tasks where the modification does not help, rather than presenting only successful examples.

Expected output: A complete results table and an evidence-based comparison of the three conditions.

A modification is not assumed to improve performance. A carefully evaluated negative result is acceptable.

Part 3: Error Analysis and Presentation

This phase examines whether reflection changes actual behavior and communicates the findings.

6. Failure-to-Retry Analysis

Analyze 5–10 pairs of consecutive attempts from initially unsuccessful tasks, where available.

For each pair, compare the failed interaction, the intervening reflection, and the next attempt. Distinguish among the following situations:

- **Incorrect reflection:** The reflection misidentifies the problem or introduces an unsupported assumption.
- **Correct reflection without correction:** The reflection identifies the problem, but the next attempt does not follow the proposed change.
- **Behavioral correction:** The next attempt changes the relevant actions, whether or not the entire task ultimately succeeds.

Include contrasting outcomes when observed. Do not invent examples to fill categories.

Expected output: A case-analysis table with brief supporting excerpts from the interaction records.

7. Final Report and Demonstration

Submit the reproducible project package, a concise report, and a short demonstration.

State exactly which components were reused and which were modified. Conclude whether reflection helped under the tested conditions, identify limitations, and propose one small future experiment.

Expected output: The materials specified in the Deliverables section below.

A carefully evaluated negative result is acceptable. Assessment follows the unified course evaluation format.

Shared Experimental Protocol

Use one fixed instruction-tuned model and the existing Reflexion implementation. Select 10 development tasks and 30 held-out evaluation tasks before examining agent performance. Keep their task IDs fixed across all conditions and document the selection rule. The conclusions concern this subset, not the full ALFWorld benchmark.

- **Condition A:** No Reflection. Each retry starts from the original environment state with no reflection memory carried between attempts.
- **Condition B:** Original Reflection. Use the existing Reflexion reflection prompt and memory mechanism.
- **Condition C:** Student-Designed Reflection. Change only the reflection prompt; retain the same action-generation prompt, examples, model, decoding settings, and memory mechanism as Condition B.

Allow at most three attempts per task and 30 environment actions per attempt in every condition. Stop retrying after success. Reset the environment to the same initial state before each retry. Conditions B and C may retain only the text memory supported by the original implementation; environment state is never carried over.

Fix the context limit, output-token limits, decoding settings, and seed schedule during the pilot and use the same settings in all conditions. Tune the student reflection prompt on development tasks only. Freeze it before evaluating the held-out tasks.

Report success on the first attempt and by the third attempt, recovery among tasks that fail on the first attempt, and the number of tasks in each denominator. Report model calls, input/output tokens, and GPU time for the complete trajectories, including reflection generation. Match first-attempt settings across conditions and record the realized initially failed task sets; do not interpret recovery rates from different failure sets as a paired comparison.

Deliverables

Submit runnable code, environment and model configurations, fixed task IDs and seeds, all prompts, complete interaction records, the comparison table, the failure-to-retry analysis, a concise report, a short demonstration, a poster, and individual contribution statements. Include the GPU-use log.

Related Work and Open Source

[1] Shridhar et al. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. ICLR 2021. ALFWorld repository

[2] Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. ReAct repository

[3] Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023.

GPU requirement

Hardware: 1 × NVIDIA A6000.

Proposed allocation cap: 30 GPU-hours per team for the complete project. This is a planning limit, not a measured runtime estimate.

No model training is required. The allocation covers development, all three evaluation conditions, retries, reflection generation, failed runs, and the demonstration. Use a short end-to-end pilot to confirm that the fixed model and task protocol fit the allocation.

Project 4

Project Title: Steering LLM Agents for Safer Tool Use

Project manager: Xiaqiang Tang

Email: xtang771@connect.hkust-gz.edu.cn

Project Description

Large Language Model (LLM) agents can interact with external tools, APIs, files, and environments. As a result, agent safety depends not only on the final natural-language response, but also on intermediate decisions such as whether a tool should

be called, whether an action is authorized, and whether additional verification is required before execution.

This final project investigates whether activation steering can make an LLM agent behave more safely during tool use. Students will build on the open-source EasyEdit2 implementation rather than develop a new steering framework. In particular, students will construct a safety steering vector from contrasting examples of safe and unsafe behavior, inject the vector into selected hidden states during inference, and evaluate whether this intervention reduces unsafe agent actions on Agent-SafetyBench.

Activation steering modifies model behavior without fine-tuning the model parameters. Given pairs of examples describing similar situations but producing different behaviors, students will collect the model's internal activations and compute a direction corresponding to safer behavior.

Can an activation steering vector reduce unsafe tool-use decisions while preserving the agent's ability to complete legitimate tasks?

Students will reproduce the existing steering workflow, construct a small safety vector, and compare it against simple baselines under a controlled evaluation protocol. The project does not require model fine-tuning, training a new reward model, modifying Agent-SafetyBench, or implementing activation steering from scratch.

Part 1: Basic Reproduction

This phase familiarizes students with tool-using agents, Agent-SafetyBench, and the EasyEdit2 steering implementation.

1. Environment Familiarization

Run the supplied agent environment and a small number of Agent-SafetyBench examples.

Explain why safety evaluation for agents differs from evaluating only the final text response. For example, an agent may produce a harmless final message after already issuing an inappropriate tool call.

Expected output:

A short interaction record from one benchmark example, accompanied by a brief explanation of the agent input, tool interface, action decision, and benchmark outcome.

Students should use the provided Agent-SafetyBench tasks rather than create new benchmark tasks or modify the benchmark evaluator.

2. Baseline Reproduction

Select one supported open-source instruction-tuned LLM and a fixed development subset of Agent-SafetyBench.

Run the following two baseline conditions:

- **Condition A:** Standard Agent — the original model and agent prompt without additional safety intervention.
- **Condition B:** Safety Prompt — the same agent with an additional system-level instruction emphasizing safe and authorized tool use.

Use the same model, decoding parameters, tool definitions, task subset, and interaction limits for both conditions.

Students are not expected to reproduce the exact numbers reported in Agent-SafetyBench. The objective is to reproduce the evaluation workflow and obtain a working baseline for later comparison.

Expected output:

Baseline benchmark results for Conditions A and B, together with several complete interaction trajectories demonstrating both safe and unsafe behavior.

3. Steering Mechanism Inspection

Run the supplied EasyEdit2 activation-steering example and trace how an activation vector is created and applied.

Students should identify:

1. which transformer layer is selected;
2. which hidden representation is extracted;
3. how activations from contrasting examples are combined;
4. where the steering vector is added during inference;
5. how the steering strength (α) affects the intervention.

Verify that activation steering changes intermediate hidden states without updating the original model weights.

Students should also inspect one example with steering enabled and disabled and compare the generated outputs.

Expected output:

An annotated example showing the original activation-steering pipeline, including the selected layer, steering-vector shape, intervention strength, and outputs with and without steering.

Students should reuse EasyEdit2's existing steering implementation rather than implement transformer hooks or activation editing from scratch.

Part 2: Safety Steering Design and Evaluation

This phase focuses on constructing one simple safety direction and evaluating it through a controlled comparison.

4. Constructing a Safety Steering Vector

Construct a small dataset of paired examples representing contrasting agent decisions.

Each pair should describe the same or closely matched situation but contain:

- a safe behavior, where the agent checks authorization, requests missing information, avoids an unsafe action, or chooses an appropriate tool action;
- an unsafe behavior, where the agent performs or proposes an inappropriate tool action.

Examples should focus primarily on agent decisions and tool use, rather than generic harmful versus harmless text.

Students may select one reasonable layer using development examples. A small comparison between a few candidate layers is permitted, but an exhaustive layer search is not required.

The model weights must remain unchanged.

Expected output:

The final paired-example dataset, selected layer, generated safety vector, vector-generation configuration, and a brief explanation of why the examples are expected to represent safer agent behavior.

Students should develop the examples and choose steering settings using development tasks only.

5. Controlled Comparison

Evaluate the following three conditions on the same fixed benchmark subset:

- **Condition A:** Standard Agent

- **Condition B:** Safety Prompt
- **Condition C:** Activation Steering

For Condition C, inject the learned safety vector during the agent's model calls using EasyEdit2. The goal is not simply to maximize refusal. A successful intervention should improve safety while preserving useful agent behavior.

Inspect at least five trajectories whose behavior changes after steering, where available; otherwise report and analyze all observed changed trajectories.

Expected output:

A development table comparing the selected steering settings, a held-out comparison of Conditions A, B, and C using the final selected setting, representative changed trajectories, and an evidence-based discussion of the safety–utility trade-off.

A steering vector is not assumed to improve the benchmark. A carefully evaluated negative result is acceptable.

Part 3: Error Analysis and Presentation

This phase examines how activation steering changes agent decisions and communicates the findings.

6. Tool-Decision Change Analysis

Analyze approximately 5–10 benchmark cases where activation steering changes the agent's behavior, where available.

For each case, compare the unsteered and steered trajectories and identify which of the following occurred:

- **Unsafe action prevented:** The original agent selected an unsafe action, while the steered agent avoided it.
- **Safer intermediate decision:** The agent added an appropriate check, verification step, authorization request, or safer tool choice.
- **No effective behavioral change:** The internal or textual response changed, but the relevant tool-use behavior remained the same.
- **Over-steering:** The intervention caused unnecessary refusal, excessive caution, or failure on a legitimate task.
- **New failure introduced:** Steering changed the trajectory but produced another incorrect or unsafe behavior.

For each example, identify where possible whether the difference appears before, during, or after the tool-selection decision.

Do not select only successful examples and do not invent examples to fill categories.

Expected output:

A case-analysis table containing short excerpts from the original and steered trajectories, the resulting tool decisions, and the assigned behavioral category.

7. Final Report and Demonstration

Submit the reproducible project package, a concise report, and a short demonstration.

Clearly state:

- which components were reused from EasyEdit2 and Agent-SafetyBench;
- which components were implemented or modified by the students;
- how the safety vector was constructed;
- which transformer layer and steering strengths were used;
- whether steering improved benchmark safety;
- whether steering reduced legitimate task completion;
- major failure cases and limitations.

The demonstration should show at least one Agent-SafetyBench case running with steering disabled and enabled, including the model's intermediate tool-use decision.

Conclude whether activation steering provided a useful safety intervention under the tested conditions and propose one small future experiment.

Expected output:

The materials specified in the Deliverables section below.

A carefully evaluated negative result is acceptable. Assessment follows the unified course evaluation format.

Optional Extension: Decision-Point Steering

The standard experiment applies the safety vector to all relevant model calls during an agent trajectory.

As an optional extension, apply steering only when the model is making a tool-selection or action-selection decision.

Compare:

- steering during every model call;
- steering only during tool/action selection.

The objective is to investigate whether targeted steering can preserve more of the base model's normal behavior while maintaining similar safety improvements.

The decision to activate steering must depend only on information naturally available during inference, such as whether the current prompt requests the next tool action. Students must not use benchmark safety labels or evaluation answers to determine when steering is activated.

Expected output:

A small additional comparison between full-time steering and decision-point steering, accompanied by an analysis of safety, legitimate task completion, and representative trajectories.

Shared Experimental Protocol

Use one supported open-source instruction-tuned model, 20 fixed development examples, and 50 held-out Agent-SafetyBench examples. Select and record the subsets before inspecting model outcomes. Keep the tool definitions and the benchmark evaluator unchanged, and use the same benchmark-supported interaction limits and decoding settings across all conditions.

Construct at most 50 safe/unsafe example pairs using training or development material only. Test at most two candidate layers and three nonzero steering strengths on a fixed subset of up to 10 development examples. Zero steering is covered by the standard-agent baseline. Choose one layer and one strength on development data and evaluate that final setting once on the held-out subset; an exhaustive layer or strength sweep is not required.

Use the benchmark-reported safety and task-completion measures. Record model calls, generated tokens, GPU time, and any evaluator expenditure. Report the subset selection and acknowledge the limited precision of small-sample comparisons.

Deliverables

Submit runnable code, pinned configurations, model and task identifiers, the paired-example dataset, steering vector, prompts, development and held-out results, complete trajectories, the case-analysis table, a concise report, a short demonstration, a poster, individual contribution statements, and a GPU-use log.

Related Work and Open Source

[1] Wang et al. EasyEdit: An Easy-to-use Knowledge Editing Framework for Large Language Models.

EasyEdit / EasyEdit2 open-source repository and activation-steering implementation.

[2] Agent-SafetyBench: Evaluating the Safety of LLM Agents.

Students should use the official benchmark implementation and evaluation protocol.

[3] Rimsky et al. Steering Llama 2 via Contrastive Activation Addition.

Introduces Contrastive Activation Addition (CAA), where activation differences from contrasting examples are used to construct behavioral steering directions.

Students are expected to reuse existing open-source implementations.

GPU Requirement

Hardware: 1 × NVIDIA A6000.

Proposed allocation cap: 30 GPU-hours per team for the complete project. This is a planning limit, not a measured runtime estimate.

No model fine-tuning is required. Include activation collection, steering-vector construction, development selection, all final conditions, failed runs, and evaluation. Confirm compatibility and memory use with a complete steering-and-evaluation pilot.

Decision-point steering is optional and must fit within the confirmed allocation; it carries no separate project-specific bonus.

Project 5

Project title: Evolving Graph-Structured Skills for LLM Agents

Project manager: Jiujiu Chen

Email: jchen349@connect.hkust-gz.edu.cn

Project description

LLM-based agents use language models to coordinate reasoning, actions, and feedback when solving tasks. To reuse effective procedures across tasks, agents can maintain reusable skills: explicit instructions or executable programs that can be retrieved and combined. However, existing skills may be incomplete or fail on unfamiliar cases, motivating skill updates that refine procedures using evidence from previous attempts. Feedback-driven approaches demonstrate how agents can improve through external reflective context without changing model parameters. Building on these ideas, this project uses GraphSkillEvo to investigate how graph-structured skills evolve, why they still fail, and which feedback and failure cases should guide subsequent updates.

Codebases and datasets

- **Baseline:** GraphSkillEvo
- **Datasets:** SearchQA (GraphSkillEvo split) and HotpotQA (provided)

Tasks

Part 1: Understand and reproduce the method

Run GraphSkillEvo using one fixed model throughout the comparison. The submitted project proposes an open-source model such as qwen3.5-9B. Initialize skills, execute the bounded evolutionary optimization loop, select the final skill using validation data, and evaluate it on fixed subsets of the provided SearchQA and HotpotQA test sets.

1. Explain the Roles of the Training and Validation Sets

- What information is obtained from training examples, and how does it influence skill updates?
- How are validation results used to rank candidates, select parents and survivors, and choose the final skill?
- Why are separate training and validation sets necessary? What risks arise from repeatedly selecting skills on the same validation set?

2. Report Results on the Provided Datasets

- Report Exact Match (EM), F1, and latency on the two fixed test subsets. Describe answer extraction and normalization, and clearly identify the evaluated subsets.
- Compare with initial-skill and no-skill baselines, analyze their difference in test cases.

3. Trace and Visualize Skill Evolution

- Visualize the skills as a flowchart produced in each iteration, starting from initialization and ending with the selected final skill.
- The flowchart should show the task-type routing, nodes, execution order, and any explicit branches or conditions.
- Discuss which changes appear helpful, ineffective, or harmful, using validation results and recorded examples.

Part 2: Analyze Skill Failures

During method development, construct the complete set of failed development examples on which the baseline selected skill receives EM = 0. Use these failures to

investigate shared problem patterns, distinct groups, and other meaningful commonalities. Do not use held-out test failures to design the optimizer modification.

After all methods and settings are frozen, construct the complete failure inventory for each fixed test subset as a final descriptive analysis. Test failures must not be fed back into skill optimization or prompt selection.

The analysis can include:

- **An error inventory:** example ID, question, reference answer(s), predicted answer, EM/F1, assigned category, and a short diagnostic explanation.
- **A failure taxonomy:** define your categories, report their counts or proportions, and explain how you handle overlapping or uncertain labels.
- **A skill-level diagnosis:** identify missing, ambiguous, ineffective, or unfollowed instructions and explain why the final skill does not resolve the case. Separate evidence-supported explanations from hypotheses.

Part 3: Redesign the skill optimization procedure

Based on the development-set analysis in Part 2, propose one concrete update to GraphSkillEvo's iterative skill optimization procedure. Choose either the optimization inputs or the failure-case selection policy as the primary modification, keeping the other component fixed. Evaluate the frozen modification against the original optimizer on the same held-out test subsets with matched model-call and generation-token budgets. Do not merely rewrite the final skill by hand.

- **Option A — Update the optimization inputs:** Specify what information should be added for the optimizer. Discuss whether successful examples should also be included to help preserve existing capabilities.
- **Option B — Design the failure-case selection policy:** Define an implementable policy for constructing each iteration's optimization batch. Address the source pool, selection criteria, diversity, difficulty, redundancy, persistent failures, and how the pool is refreshed after updates. Explain why your policy should be more informative than the baseline policy you inspected.

Optional Extension: Cross-Dataset Skill Generalization

Choose at most one extension after completing the required comparison and only within the confirmed allocation:

- **Option A:** Transfer Skill Capabilities from SearchQA to HotpotQA. Use the evolved SearchQA skill as the starting point for adaptation to HotpotQA. Identify

reusable capabilities and refine or extend them using HotpotQA training feedback. Compare against both the unadapted SearchQA skill and a skill independently evolved on HotpotQA to assess the benefits of capability transfer.

- **Option B:** Jointly Update Skill Capabilities across SearchQA and HotpotQA. Iteratively update one shared skill using feedback from both datasets. Design how examples and failures are combined, resolve conflicting updates, and evaluate whether joint updates improve complementary capabilities while preserving performance on both tasks.

Part 4: Finish the final report and demonstration

Write a report of up to six pages covering research background, research problem, methodology, experiments, and conclusions. Implementation details may be included in an appendix. Submit the code, fixed data IDs, prompts, skill graphs, evaluation outputs, comparison tables, GPU accounting log, and demonstration materials required by the unified course format.

References

[1] GraphSkillEvo: Evolutionary Optimization of Graph-Structured Agent Skills. arXiv, 2026.

[2] SkillOpt: Executive Strategy for Self-Evolving Agent Skills. Microsoft. 2026

[3] Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023.

[4] ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.

GPU hours

Hardware for local inference: 1 × NVIDIA A6000.

Proposed allocation cap: 40 GPU-hours per team for the complete project. This is a planning limit, not a measured runtime estimate. No model-parameter training is required.

For each dataset, use at most 100 training examples for skill updates, 30 validation/development examples for selection and diagnosis, and 50 held-out test examples. Preserve the supplied partition boundaries. Select fixed IDs before inspecting outcomes, and disclose repeated validation use as a limitation.

Use at most three evolution rounds and at most two newly evaluated candidates per round, in addition to any retained current best. Fix execution and generation limits in the pilot. Apply the same limits to the original and modified optimizer, and retain the initial-skill and no-skill baselines.

Cache identical deterministic data access or tool results where valid, but do not count reused model outputs as fresh independent runs. Account for calls used to create, execute, score, and revise skills, not just final test inference.

Project 6

Project title: Prompting and Fine-tuning Language Models for Grid Path Planning

Project manager: Yue Chang

Email: ychang500@connect.hkust-gz.edu.cn

Project description

This project studies whether language models can detect and repair errors in grid-based navigation plans. It builds on the PPNL benchmark introduced in “Can Large Language Models be Good Path Planners?”, where an agent must move from a start location to a goal while avoiding obstacles.

Instead of evaluating only direct path generation, this project introduces a generate–verify–repair pipeline. First, a language model generates or receives a candidate action sequence. A verifier must determine whether the sequence is feasible, identify the first failure location, and classify the failure type, such as moving out of bounds, hitting an obstacle, or stopping before reaching the goal. A repair model must then produce a corrected action sequence using the grid description, the candidate plan, and the verifier feedback.

Training examples will be constructed automatically from valid shortest paths by introducing controlled errors such as action substitution, action deletion, premature stopping, or multiple errors. All generated examples will be checked using the official single-goal executor. The project will compare direct generation, self-verification, and executor-guided repair, and investigate whether models trained on single-error plans can generalize to plans containing multiple errors.

For a feasible undergraduate NLP project, the scope will remain limited to single-goal environments. Students may use prompting or fine-tuning with small encoder-decoder models such as Flan-T5-small, Flan-T5-base, or BART-base. The main research

question is whether explicit verification and feedback can improve the feasibility and success rate of language-model-generated plans.

Codebases and datasets

- **Baseline:** T5-small checkpoint (source link)

T5-base checkpoint (source link)

BART-base checkpoint

- **Benchmark:** PPNL benchmark repository
- **Additional Data:** Verification and repair examples will be generated from the official single-goal environments and shortest paths. No external dataset or Gurobi dependency is required.

Settings

1. Benchmark: PPNL-derived Plan Verification and Repair Benchmark.

2. Baselines: Flan-T5-small, Flan-T5-base, and BART-base. Compare at least two models in the in-distribution baseline study, then select one primary model using development results for the complete verification, repair, improvement, and ablation experiments. The second model does not need to repeat the full study.

3. Scope: Use the single-goal setting only. The multi-goal pipeline is excluded because it is more difficult to reproduce and may require Gurobi.

4. Environment and data:

- Use 6×6 environments for training and in-distribution testing.
- Split the data by grid environment rather than by individual action sequence, so that paths from the same grid do not appear in both training and testing sets.
- Include both optimal paths and feasible but non-optimal paths as valid examples.
- Construct invalid examples using controlled action substitutions, deletions, insertions, or premature stopping.
- Use single-error candidates for training and in-distribution testing.
- Use candidates containing two independently introduced errors as the required out-of-distribution evaluation set.
- A 7×7 or denser 6×6 evaluation set may be included as an optional additional experiment.

5. Tasks:

- **Plan generation:** Given a natural-language grid description, generate an action sequence in the format up / down / left / right.
- **Plan verification:** Given a grid and a candidate action sequence, predict whether it is valid. For an invalid plan, also predict the first failure step and failure type.
- **Plan repair:** Given a grid, a candidate plan, and verifier feedback, generate a corrected action sequence.
- **End-to-end evaluation:** Compare direct generation with generate–verify–repair.

6. Verification output format:

- **Valid plan:** valid
- **Invalid plan:** invalid | step=<index> | reason=<error_type>
- Error types should include at least obstacle collision, out of bounds, invalid action token, and goal not reached.
- The official executor should be treated as the source of ground-truth labels.

7. Evaluation metrics:

- **Generation and repair:** Success Rate, Feasibility, and Optimality.
- **Verification:** Valid/invalid Macro-F1 and first-error localization accuracy.
- **Repair Success Rate:** Percentage of invalid candidate plans that become successful after repair.
- **End-to-End Success Rate:** Percentage of environments solved by the complete generate–verify–repair pipeline.
- Students should also report results separately for different failure types.

8. Required comparisons:

- Direct generation without repair.
- Generation followed by self-verification and repair.
- Generation followed by executor feedback and repair.
- In-distribution single-error test set versus out-of-distribution multiple-error test set.

Part 1: Reproduce the Basic Algorithm

1. Correct implementation of the single-goal executor and automatic construction of valid and corrupted candidate plans.

2. Correct implementation of direct plan generation and verification/repair baselines.

3. Explore and compare at least two baseline models, such as Flan-T5-small, Flan-T5-base, or BART-base, on the same in-distribution 6×6 test set. Select the primary model for Parts 2 and 3 using development results only, not test performance.

Part 2: Modify the Core Module / Reasoning Schema

1. Investigation of failure cases

Provide detailed execution traces for representative failures. Examples include incorrectly accepting an invalid plan, reporting the wrong failure step, repairing only the first of multiple errors, introducing a new collision during repair, or producing a feasible but unnecessarily long path.

2. Implement the generate–verify–repair reasoning schema

Build a pipeline in which a verifier diagnoses a candidate plan and a repair model uses the diagnosis to produce a new plan. Compare self-generated verifier feedback with feedback produced by the symbolic executor.

3. Propose an improved verification or repair method

Possible approaches include structured state-tracking prompts, Chain-of-Thought reasoning, iterative repair, verifier-guided decoding, constrained action decoding, multitask learning, or training-data augmentation. The proposed method should be evaluated on both single-error and multiple-error test sets using the primary model and should include one focused ablation study. Repeating the improved method and ablation with the second model is optional.

Part 3: Report and Presentation

1. Written report with academic quality

The report should contain an abstract, introduction, related work, methodology, experimental setup, results, error analysis, ablation studies, and conclusion. It must report Success Rate, Feasibility, Optimality, verification Macro-F1, first-error localization accuracy, and Repair Success Rate on one in-distribution and one out-of-distribution test set. At least two detailed repair case studies should be included.

Related Work

(1)PPNL Benchmark: a controlled benchmark for spatial-temporal reasoning in grid navigation.

(2)Chain-of-Thought and ReAct: prompting strategies that encourage explicit reasoning and interaction with feedback.

(3)Self-Refine and Reflexion: approaches that use feedback to revise model-generated outputs.

(4)Verifier-guided and neuro-symbolic generation: combining learned generation with symbolic validation.

(5)T5, Flan-T5, and BART: encoder-decoder language models for text-to-text generation and structured prediction.

GPU hours

Hardware: 1 × NVIDIA 4090.

Proposed allocation cap: 20 GPU-hours per team for the complete project. This is a planning limit, not a measured runtime estimate.

Use at most 4,000 training examples in total across generation, verification, and repair, plus up to 200 development examples. Use up to 200 examples in each of the required single-error and multiple-error test sets. Maintain the environment-level split and record the construction rules and IDs. CPU-based data construction and executor checks should be logged separately from GPU use.

Use one fixed seed and a small development-selected training configuration, with at most three passes over each applicable training subset. Compare the two baseline models with matched data and training budgets. Run the full pipeline, one improvement, and one ablation only for the primary model.

The pilot must verify the exact model checkpoint, input-length handling, memory use, and training/evaluation throughput. The submitted source lists T5 repository links while naming Flan-T5 in the model options; the project manager must confirm the intended checkpoint before students start. Do not silently treat T5 and Flan-T5 as the same checkpoint.

Additional grid sizes, dense environments, more seeds, or a second-model improvement study are optional and must fit within the confirmed allocation.

Project 7

Project title: Distributional Critics for Credit Assignment in Tool Assisted Reasoning

Project manager: Zhaofan Zhang

Email: zzhang398@connect.hkust-gz.edu.cn

Project description

Language-model agents must decide when to call a tool, which arguments to provide, and how to use the returned information. A final correctness reward does not directly identify which earlier decisions contributed to success or failure. This project studies whether a distributional critic provides useful learning signals for these decisions under a limited computational budget.

Students will build on DisPPO, published at ICML 2026, which combines a quantile-based critic with Distributional Generalized Advantage Estimation (DGAE). The project adapts this method to a small language model solving GSM8K problems with a calculator. Students will reproduce PPO and DisPPO in this reduced setting, inspect their credit-assignment signals, and evaluate one focused modification to critic training.

The central question is whether distributional value estimation improves learning and early-decision feedback in short tool interactions. This is a course-scale reproduction and adaptation of the method; matching the paper's large-model benchmark scores is not required.

Objectives

- Understand the quantile critic, its training targets, and how DGAE supplies advantages to the policy update.
- Compare scalar and distributional critics using the same model, task reward, tool interface and sampling budget.
- Investigate value prediction and advantage signals at different interaction stages.

- Implement and evaluate one small change, with conclusions supported by controlled experiments and trajectory analysis.

Task and Computational Scope

Use Qwen2.5-0.5B-Instruct and one CPU-based calculator supporting basic arithmetic. The agent may make at most three tool calls and generate at most 512 assistant tokens in total per episode. The main task uses short GSM8K examples. The required comparison uses one shared SFT checkpoint and one RL seed per trained condition on a single A6000; detailed scope and the proposed allocation are specified below.

Part 1: Basic Reproduction

1. Environment and data preparation

Run a small plain-GSM8K PPO smoke test using the single-GPU verl example. Prepare fixed train, development and test partitions, then implement the calculator interface. Convert and execution-check a small set of training-only calculation annotations for tool-format SFT.

2. Baseline reproduction

Compare Condition A, the shared SFT model without RL; Condition B, PPO with a scalar critic and GAE; and Condition C, DisPPO with a quantile critic and DGAE. B and C use the same warm-start policy, reward, interaction limits and total rollout budget. Save complete trajectories and training logs.

3. Mechanism inspection

Trace one trajectory through tool execution, reward assignment, critic target construction and advantage computation. Explain the token masks and verify that tool observations are excluded from the policy loss while return propagation remains correct.

Expected output: A runnable baseline, fixed data manifests, a comparison table, and an annotated trajectory linking observations, actions, rewards, value estimates and advantages.

Part 2: Credit Analysis and One Modification

4. Credit-signal diagnosis

Compare value-prediction error and advantage statistics at early, middle and final assistant turns. At a fixed checkpoint, freeze the evaluated policy and sample a small number of continuations from development prefixes. Use these outcomes to assess predictions under that policy. Limit this diagnostic to one selected checkpoint per trained condition, at most 20 development prefixes, and up to four continuations per prefix.

5. Focused modification

Implement Condition D by balancing the critic loss across assistant turns. First average valid per-token critic losses within each turn; then average across turns in a trajectory and across trajectories. This gives short tool-decision turns comparable total weight to long answer turns. Keep the policy loss, reward, quantile targets and DGAE settings unchanged.

6. Controlled comparison

Compare C and D using matched model initialization, seed allocation, optimizer steps, rollout counts and token limits. Use fresh on-policy rollouts for each method; do not reuse another trained policy's trajectories as if they were on-policy. Report whether the modification helps early decisions or harms other parts of the trajectory.

Expected output: A decision-stage analysis, the modified loss implementation, a controlled C-versus-D comparison, and an explanation of improvements, regressions or null results.

Part 3: Failure Analysis and Communication

7. Failure analysis

Inspect 5–10 paired examples across methods. Distinguish incorrect mathematical plans, invalid tool arguments, incorrect use of tool results, unnecessary calls and truncated answers.

8. Report and demonstration

Submit the reproducibility package and a concise demonstration. State what was reused and modified, and describe the limits of the conclusions.

A carefully executed negative result is acceptable. Assessment follows the unified course evaluation format, with no project-specific percentage allocations.

Shared Experimental Protocol

Dataset and Tool Trajectories

Use the GSM8K main configuration. Define a short-example selection rule before training, using reference-solution length and the number of executable calculation annotations. Report the rule and retained proportion. Reserve 100 development questions from the official training split, use up to 1,000 remaining questions for RL, and construct up to 300 SFT demonstrations from the training partition only. Any SFT–RL overlap must be documented.

Apply the same fixed selection rule to the official test split and sample up to 100 questions before experimentation. These results describe the retained short-example subset, not full GSM8K. Never filter test questions using a model's success or failure. Save the selected IDs and random seeds.

Convert validated calculation annotations into calculator calls and observations. Permit only a restricted arithmetic grammar. At evaluation time, the calculator receives the agent's expression only; it must not receive the reference answer or indicate whether a proposed answer is correct. Invalid calls return a fixed error message and count toward the three-call limit.

Required Conditions

- **Condition A:** SFT only. No RL after the shared warm start; measure the starting policy.
- **Condition B:** PPO. Scalar critic and GAE; establish the scalar baseline.
- **Condition C:** DisPPO. Quantile critic and DGAE; assess the distributional method.
- **Condition D:** Modified DisPPO. C with critic loss balanced across turns; assess one focused change.

Training Controls

Use the same 0.5B backbone for actor and critic, one RL seed per trained condition, and a shared SFT checkpoint. The required RL experiments are one run each for B, C, and D. Cap initial prompts at 512 tokens and the full interaction context at 1,536 tokens. Start with 16 prompts per rollout batch, two responses per prompt, and micro-batches of one or two trajectories. Use a common budget of 50–100 iterations, fixed after the pilot and before the main comparison.

Measure memory and end-to-end throughput, including rollout generation and critic/policy updates, before fixing the configuration. If the planned workload exceeds

the proposed allocation, reduce the common iteration budget for B, C, and D before the main runs. Do not give one method extra optimization or generation budget. Results describe a small, single-seed course-scale experiment rather than a statistically stable ranking.

Use 51 quantiles, mean aggregation and the bounded delta critic head from the DisPPO math recipe for the correctness-only task. The published ReTool recipe uses a different head; record the selected implementation explicitly. Disable optional exploration bonuses and dynamic group filtering in the initial comparison. Keep any KL regularization in the actor objective, outside the task-return targets, consistently across methods.

Set gamma to one and use the same GAE lambda across B, C and D. The final-answer reward is one for a correct numeric answer and zero otherwise, including missing answers and truncation without a valid final answer. Intermediate arithmetic correctness receives no reward. Terminal bootstrapping must be zero. Keep the critic's support consistent with the task return. Use development data for tuning and checkpoint selection; evaluate the frozen final choices on the test set.

Evaluation and Interpretation

- Report final-answer accuracy, valid tool-call rate, calls per successful episode, truncation rate and learning curves against both cumulative rollout tokens and GPU time. Include the SFT-only baseline.
- Report value-prediction error by assistant turn and advantage statistics. For comparable value diagnostics, use the mean of the quantile predictions. Report the required single-seed result explicitly and do not claim training stability or statistical significance from it. If the optional second seed is completed for all trained conditions, show both results and their range; two seeds still support only limited claims about stability.
- Inspect a small paired intervention study on development examples: change one proposed call, execute the changed expression, and continue with the same frozen policy. Compare the outcome change with the predicted learning signal. Use at most 10 development examples and keep this diagnostic within the evaluation allocation.

Binary terminal returns are Bernoulli and are determined by their success probability. Gains cannot by themselves demonstrate access to additional higher-order return information. Quantile spread is not automatically epistemic uncertainty or causal

importance. The proposed turn weighting is a new course extension; a claim that it helps distributional critics more than scalar critics requires an additional matched scalar-critic control.

Final Report and Deliverables

Submit a report of up to six pages, excluding references and appendices, covering motivation, methods, experimental setup, results, credit-signal analysis, failures and limitations. Include runnable code, pinned dependencies and repository commit, configuration files, split IDs, seeds, the calculator interface, SFT-conversion rules, evaluation outputs, training plots, selected trajectories, a GPU accounting log, a poster and individual contribution statements.

Base Paper and Reproduction Source

[1] DisPPO: Quantile-Based Distributional Reinforcement Learning for Large Language Models. ICML 2026. This is the required methodological basis.

[2] Paper link retained from the submitted draft

Model and Dataset Sources

Qwen2.5-0.5B-Instruct checkpoint

GSM8K repository

GSM8K dataset

The original GSM8K calculator sampler is an illustrative implementation and is not optimized for batched generation. Reuse its annotation convention and arithmetic logic rather than its unbatched sampling loop.

GPU requirement

Hardware: 1 × NVIDIA A6000 with 48 GB memory.

Proposed allocation cap: 50 GPU-hours per team for the complete project. This is a planning limit, not a measured runtime estimate.

Include shared SFT, the PPO smoke test, all three RL runs and fresh on-policy rollouts, tuning, failed runs, diagnostics, the intervention study, and final evaluation. Count rollout time as well as optimizer time.

Optional Extension

An optional second RL seed must cover B, C, and D and fit within the confirmed cap. A matched scalar-critic turn-weighting control is also optional; without it, do not claim a greater benefit for distributional critics. Neither extension carries a separate bonus.

Project 8
Deep Research Agents from Error Localization to Targeted Repair

Project manager: Linxiao Cao
Email: lcao950@connect.hkust-gz.edu.cn

Project description

Deep Research Agents solve complex information-seeking tasks through long trajectories involving search, tool use, evidence inspection, reasoning, and answer synthesis. Existing evaluation typically focuses on the final report, but a poor final answer does not reveal where the research process first became unreliable. An incorrect intermediate claim may propagate through later reasoning, while other mistakes may be harmless because the agent subsequently recovers.

Recent work, **Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories**, studies this problem at the trajectory level and introduces TELBench and DRIFT for identifying harmful error spans and the first consequential error in Deep Research trajectories. The released benchmark contains annotated trajectories that distinguish harmful errors from normal exploration, failed searches, and other non-consequential behavior.

This project investigates a natural follow-up question:

Can identifying the first consequential error help a Deep Research Agent repair a failed trajectory more effectively than simply revising the final answer or repeating the entire research process?

Students will reproduce an existing trajectory error-localization method, study how errors propagate through Deep Research trajectories, and design a focused repair method based on localized errors. The goal is to understand the relationship between **error localization, error propagation, and repairability**, rather than simply improve final benchmark scores.

Tasks

Part 1: Reproduce and understand trajectory error localization — 30%

Students are **not** required to reproduce the exact performance reported in the original paper. Successfully running the core error-localization pipeline with an open-source model and reporting results on a fixed evaluation set is sufficient for the reproduction component. Any differences from the original experimental setup should be documented.

Reproduce the basic error-localization pipeline from **Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories.** Use separate development and test sets for method development and final evaluation, and report macro Precision, Recall, F1, and First-Error Accuracy.

Evaluate whether harmful trajectory spans and the first consequential error can be identified reliably. Inspect successful and unsuccessful examples and explain how harmful errors differ from unsuccessful but non-consequential exploration.

Analyze where errors occur in the research process, such as retrieval, evidence interpretation, reasoning, or answer synthesis.

Expected output: A runnable reproduction, quantitative localization results, and representative annotated trajectories.

Part 2: Analyze error propagation in Deep Research — 25%

Investigate how localized errors influence subsequent steps of a research trajectory.

Study whether later reasoning corrects an earlier error, ignores it, or builds additional claims on top of it. Analyze whether errors occurring at different stages exhibit different downstream effects and whether some types of errors are more likely to propagate to the final answer.

Construct a small failure taxonomy and use trajectory examples to distinguish between an intermediate error that is later recovered and one that becomes consequential.

Expected output: An analysis of error types, error propagation, and representative successful and unsuccessful recovery cases.

Part 3: From error localization to targeted repair — 35%

Based on the findings from Part 2, design a focused method for repairing Deep Research trajectories using localized error information.

Possible interventions include revising the problematic step, restarting research from the localized error, or discarding unreliable downstream reasoning. The modification should remain focused and should reuse an existing Deep Research Agent rather than introduce a new agent architecture.

Use the same base model and tool configuration across methods, and report additional inference cost, including generated tokens and tool/search calls. Investigate whether repair effectiveness depends on the location or type of error and whether inaccurate error localization limits the usefulness of targeted repair.

A targeted repair method is not assumed to outperform full retry. A carefully analyzed negative result is acceptable.

Expected output: A controlled comparison of repair strategies and an analysis of when localized repair succeeds or fails.

Part 4: Final report and demonstration — 10%

Submit a report of approximately 6 pages covering motivation, reproduction, error analysis, the proposed repair method, experiments, failure cases, and conclusions, together with runnable code and representative trajectories. Recommended sections include an abstract, related work, methods, experimental results, and conclusions.

Clearly state which components were reused and which were modified.

Codebases and datasets

Error localization: DRIFT / TELBench, released with **Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories**.

Deep Research Agent: **DR Tulu: Reinforcement Learning with Evolving Rubrics for Deep Research** provides an open Deep Research model together with an agent library and evaluation infrastructure.

Evaluation benchmark: **DeepResearch Bench: A Comprehensive Benchmark for Deep Research Agents** provides 100 expert-designed research tasks across 22 fields together with report- and citation-oriented evaluation.

Use development data for method design and hyperparameter/prompt selection. The final test set should remain frozen until final evaluation.

Related work

[1] Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories

[2] DeepResearch Bench: A Comprehensive Benchmark for Deep Research Agents

[3] DR Tulu: Reinforcement Learning with Evolving Rubrics for Deep Research

GPU hours

< 50GPU hours

No model training or proprietary LLM APIs are required. Students may use locally deployed open-source models such as **Qwen2.5-7B-Instruct** or **Qwen3-8B** with vLLM. **DR-Tulu-8B** may also be used for the Deep Research Agent component. Cached trajectories and retrieved evidence may be used to reduce computational costs

Project manager: Linxiao Cao
Email: lcao950@connect.hkust-gz.edu.cn

Project description

LLM agents can improve during deployment by using information obtained from interactions with tools and environments. However, not all test-time experience should necessarily be retained or transferred in the same way. Some experience describes the **environment itself**, such as how a tool behaves, which constraints an API imposes, or what state transitions follow from an action. Such information may remain useful across multiple tasks in the same environment. Other experience is **task-specific**, such as temporary entities, intermediate results, or an action plan for a particular user request. Reusing this information across unrelated tasks may be irrelevant or harmful.

Grounded Test-Time Adaptation for LLM Agents demonstrates that environment-specific information obtained during deployment can improve agent behavior through syntactic adaptation and dynamics grounding. The work evaluates adaptation in function-calling and interactive environments, including BFCL.

At the same time, work such as **Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory** studies agents that continually accumulate and reuse experience across task streams, highlighting the importance of memory evolution during test-time learning.

This project asks:

What information should an LLM agent retain and transfer during test-time adaptation?

Students will reproduce an existing test-time adaptation method, analyze the information acquired during adaptation, and study whether environment-level knowledge and task-specific experience exhibit different transfer properties.

Tasks

Part 1: Reproduce test-time adaptation for LLM agents — 30%

Students are **not** required to reproduce the exact performance reported in the original paper. Successfully running the core non-parametric adaptation pipeline with an open-source model and reporting baseline results is sufficient for the reproduction component. Differences from the original experimental setup should be documented.

Reproduce the non-parametric environment-grounding component of **Grounded Test-Time Adaptation for LLM Agents** on a manageable tool-use setting.

Trace how information obtained from environment interaction is converted into rules, context, or memory that influences later agent decisions. Compare the adapted agent with a corresponding agent that does not use test-time environment information.

The objective is to establish a working adaptation pipeline and understand what information the baseline actually acquires.

Expected output: A runnable reproduction, baseline results, and representative interaction trajectories.

Part 2: Analyze the structure of test-time knowledge — 25%

Inspect the information accumulated during adaptation and investigate whether it can be meaningfully separated into different types.

In particular, analyze:

environment-specific knowledge that may generalize across tasks;

task-specific knowledge that is useful only within one episode;

mixed or ambiguous information that is difficult to assign to either category.

Study whether different types of acquired knowledge exhibit different transfer behavior. Also examine incorrect or over-generalized memories that may be useful for one task but harmful when reused elsewhere.

Expected output: A taxonomy of test-time knowledge and an analysis of useful and harmful experience reuse.

Part 3: Study knowledge transfer across tasks and environments — 35%

Design a focused adaptation or memory mechanism that treats knowledge with different expected scopes differently.

Evaluate the agent on sequences of tasks from the same environment and across changes of environment. Investigate whether environment-level knowledge learned from earlier interactions improves later tasks, whether task-specific experience causes negative transfer, and whether previously useful knowledge becomes stale after an environment change.

Compare the proposed method with suitable baselines such as resetting adaptation between tasks or retaining previous experience without distinguishing its scope.

The purpose of this part is to understand **what test-time knowledge transfers, when it transfers, and when it becomes harmful**, rather than simply maximize the amount of stored memory. A well-executed negative result is acceptable.

Expected output: A controlled study of transfer, interference, retention, and failure cases across task sequences.

Part 4: Final report and demonstration — 10%

Submit a report of approximately 6 pages covering motivation, reproduction, analysis of test-time knowledge, the proposed adaptation mechanism, transfer experiments, failure cases, and conclusions, together with runnable code and representative trajectories. Recommended sections include an abstract, related work, methods, experimental results, and conclusions.

Clearly distinguish failures caused by incorrect knowledge extraction from failures caused by inappropriate knowledge transfer.

Codebases and datasets

Adaptation baseline: official implementation of **Grounded Test-Time Adaptation for LLM Agents**.

Main benchmark: Berkeley Function Calling Leaderboard (BFCL). The associated paper is **The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models**. BFCL includes stateful and multi-step function-calling evaluation and is therefore suitable for studying experience transfer in tool-using agents.

Optional memory benchmark: **Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory** provides sequential task streams and implementations of multiple memory mechanisms.

Students should use fixed task streams from selected API domains, with predefined task order and environment changes. Adaptation may use only information available

from past and current interactions. Report task success, tool-call correctness, cross-task transfer, and adaptation cost.

Use development data for method design and hyperparameter/prompt selection. The final test set should remain frozen until final evaluation.

Related work

[1] Grounded Test-Time Adaptation for LLM Agents

[2] Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory

[3] From Player to Master: Enhancing Test-Time Learning of LLM Agents via Reinforcement Learning over Memory

[4] The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models

GPU hours

< 50GPU hours

Students should use locally deployed open-source LLMs, such as Qwen2.5-7B-Instruct or Qwen3-8B, rather than proprietary LLM APIs (e.g., GPT or Claude) for experiments. Web access and external retrieval tools are permitted.

No model fine-tuning is required. Students should focus on non-parametric test-time adaptation, controlled experiments, and meaningful analysis of knowledge transfer and failure cases.

Project 10
Improving Graph-Based Retrieval for Multi-Hop Question Answering

Project manager: Ziwen Zhao
Email: zzhao681@connect.hkust-gz.edu.cn

Project description

Graph-based retrieval-augmented generation (Graph-RAG) connects information across documents, but building a graph can be costly, and the resulting connections do not always help retrieve the evidence needed for an answer. This project studies these trade-offs using Microsoft GraphRAG's lightweight implementation, nano-graphrag, and HippoRAG 2. Students will reproduce the existing systems on a small multi-hop question-answering benchmark, analyze their failures, and make one focused improvement to graph construction efficiency or retrieval accuracy. LightRAG provides additional related work on efficient graph-based retrieval. No model training or new theoretical result is required.

Tasks

Part 1: Prepare the benchmark and reproduce the baselines (40%)

Inspect HotpotQA's questions, answers, passages and supporting-evidence annotations. Create fixed development and test sets, verify that the indexed corpus contains the supporting passages for every selected question, and save all selected IDs.

Run the standard dense-RAG baseline, nano-graphrag's local retrieval mode and HippoRAG 2 on the same corpus and questions. Trace one question through indexing, retrieval and answer generation to explain what each system does.

Evaluate supporting-passage recall@5 (excluding the community summaries), the proportion of questions with all supporting passages retrieved in the top five, answer Exact Match and F1, indexing time, model-token use and query latency. Use a comparable answer-generation setup and retrieved-context budget.

Expected output: Runnable baselines, fixed dataset splits and configurations, an annotated pipeline example, and a results table with clearly defined metrics. Matching full-paper scores is unnecessary.

Part 2: Analyze costs and failure cases (15%)

Measure where indexing time and model tokens are spent. Inspect at least 20 questions, including cases where the systems disagree or all fail. Compare the supporting passages with the graph-derived information, retrieved evidence and final answer. Determine whether each failure arises during graph construction, evidence retrieval or answer generation, and select one problem to address in Part 3.

Expected output: A cost breakdown and an example-based failure inventory that explains the selected research problem.

Part 3: Implement and evaluate one focused improvement (35%)

Choose **either** graph construction efficiency **or** retrieval accuracy as the primary goal. Identify the stage of nano-graphrag to change and define the expected benefit using development data. No particular improvement technique is prescribed.

Implement one limited change. Keep other components consistent with the baseline. Rebuild the index on the same corpus if graph construction changes; reuse the baseline index when possible if only retrieval changes.

Compare the unchanged and modified systems on the frozen test set. Report relevant quality and cost metrics, and inspect examples of improvements and regressions. Discuss whether any efficiency gain reduces retrieval quality, or whether any accuracy gain requires additional computation.

Expected output: Runnable modified code, a controlled baseline-versus-modified results table, a quality–cost plot and an evidence-based analysis. A carefully analyzed negative result is acceptable.

Part 4: Final report and demonstration (10%)

Write a report of approximately 6 pages covering motivation, methods, experimental setup, results, failure analysis and limitations. Demonstrate one case by showing its retrieved evidence and answer before and after the change. State exactly which

existing components were reused and which were modified. Recommended sections include an abstract, related work, methods, experimental results, and conclusions.

Expected output: The report and demonstration, together with runnable code, configurations, contribution statements, etc.

Codebases and datasets

GraphRAG implementation: nano-graphrag (local retrieval mode), HippoRAG 2, etc.

Comparison methods: standard dense-RAG baseline, e.g., BM25, DPR, Faiss.

Dataset: A fixed subset of the repository's HotpotQA questions and passage corpus. Use approximately 1,500 passages and 300 questions, divided into development and test sets. Retain the annotated supporting passages for each selected question. Use the same corpus, generation model, embedding model and answer-evaluation procedure for all methods.

⚠**NOTE**: We do NOT strictly require matching the original paper's setup exactly for the reproduction part, so you can use local models (e.g., Llama-3.1-8B/70B-Instruct from vLLM/Ollama) to extract relation triples or answer questions. For local extraction, you may find graphrag-local-ollama useful.

Related work

[1] Edge et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. 2024.

[2] Gutiérrez et al. *HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models*. NeurIPS 2024.

[3] Gutiérrez et al. *From RAG to Memory: Non-Parametric Continual Learning for Large Language Models*. ICML 2025.

[4] Guo et al. *LightRAG: Simple and Fast Retrieval-Augmented Generation*. Findings of EMNLP 2025.

[5] Yang et al. *HotpotQA: A Dataset for Diverse, Explainable Multi-Hop Question Answering*. EMNLP 2018.

GPU hours

< 50GPU hours

Project 11 Compressing Chain-of-Thought via Interpretable Latent Reasoning

Project manager: Ziwen Zhao
Email: zzhao681@connect.hkust-gz.edu.cn

Project description

Chain-of-thought reasoning can require many generated tokens. CODI compresses a textual reasoning chain into continuous latent thoughts through self-distillation. SIM-CoT adds supervision for individual reasoning steps; SELR studies explanations decoded from latent thoughts. This project asks whether a small change to an existing post-training method can improve reasoning compression or make the information in latent steps easier to interpret, while retaining answer accuracy. Students should develop one focused improvement.

Tasks

Part 1: Understand and reproduce the baselines (20%)

Explain what CODI distills from textual reasoning and what SIM-CoT supervises at each latent step. Run the released GPT-2 checkpoints on the fixed test subsets. Record exact-match answer accuracy, the number of latent reasoning tokens, the length of the provided textual reasoning chains, and inference latency under the same evaluation settings. Then train one small control run from the GPT-2 base model using the selected training subset for at most three epochs; save its configuration and checkpoint. Results on these restricted subsets need not match paper-scale results.

Expected output: Reproducible scripts, fixed data splits, a trained control checkpoint, and one baseline results table.

Part 2: Study compression and latent-step meaning (35%)

Compare answer accuracy with reasoning length across the baselines. Use the control run's training-time decoder to inspect multiple held-out examples, including correct and incorrect answers. For each decoded step, check whether its arithmetic operation, numbers, and intermediate result agree with the provided reasoning chain. Group failures such as missing steps, incorrect operations, and correct answers with inconsistent decoded steps.

Expected output: One diagnostic analysis containing compression measurements, step-level consistency counts, and representative success and failure cases.

Part 3: Develop and evaluate one focused improvement (35%)

Choose either greater compression at comparable answer accuracy (e.g., fewer reasoning steps) or clearer latent-step meaning at a stated latent budget. Write a short hypothesis based on Part 2, implement one change within the existing CODI/SIM-CoT pipeline, and train it from the same GPT-2 initialization as the control. Keep the data, training-step budget, and evaluation procedure matched. Compare the modified model with the control on the test subsets using answer accuracy, reasoning length, latency, and the step-level analysis from Part 2. Analyze whether any gain holds across problem types and report unsuccessful results as well. Students may adopt cutting-edge post-training strategies to better improve the accuracy, e.g., Group Relative Policy Optimization.

Expected output: One controlled comparison of the proposed change and the matched control, with code, metrics, and an explanation of the observed trade-off.

Part 4: Final report and demonstration (10%)

Write a report of approximately 6 pages. Present the motivation, method, experimental settings, main results, limitations, and several decoded-step examples. Submit runnable code and configurations so that the restricted experiments can be repeated. Recommended sections include an abstract, related work, methods, experimental results, and conclusions.

Expected output: A concise project report, code, configurations, and a short demonstration.

Codebases and datasets

Choose one of the official implementations from CODI or SIM-CoT of the GPT-2 checkpoints. Select the first 2,000 training examples from GSM8k-Aug, reserve 200 separate examples for validation, and use fixed subsets of GSM8K test examples for in-distribution evaluation and GSM-Hard/SVAMP examples for out-of-distribution evaluation.

Related work

[1] Shen et al. *CODI: Compressing Chain-of-Thought into Continuous Space via Self-Distillation*. EMNLP 2025.

[2] Wei et al. *SIM-CoT: Supervised Implicit Chain-of-Thought*. ICLR 2026.

[3] Zhao et al. *Think in Latent, Explain in Language: Self-Explainable Latent Reasoning*. ICML 2026.

GPU hours

< 50GPU hours

Project 12
Speaker- and Group-Aware Memory for Multi Party Conversations

Project manager: Jindong Li

Email: jli839@connect.hkust-gz.edu.cn

Project description

Most existing long-term memory systems for LLM agents are designed for one-to-one conversations. However, real-world interactions often involve multiple participants communicating across group chats, meetings, or collaborative channels. In such

settings, remembering conversation content alone is insufficient. A memory system must also preserve who said what, in which group, and under what conversational context.

This project investigates long-term memory mechanisms for multi-party conversations using GroupMemBench and EverMemBench. Students will study whether explicitly incorporating speaker identity, group information, reply structure, and temporal information into memory retrieval can improve downstream question answering. In particular, the project focuses on failures caused by treating multi-party conversations as independent text chunks, where information from different speakers or groups can easily be confused.

The central research question is whether speaker- and group-aware memory representations and retrieval mechanisms can improve long-term memory performance in multi-party conversations compared with conventional content-based retrieval systems.

Tasks

Part 1: Reproduce basic memory retrieval in multi-party conversations (20%)

Use GroupMemBench as the primary benchmark and reproduce the two retrieval baselines provided in its official repository:

BM25 Retrieval: retrieves historical messages based on lexical similarity.

Dense Embedding Retrieval: retrieves historical messages using embedding model and vector similarity.

In the released baselines, retrieval is primarily performed over message content, while speaker and role metadata are attached to retrieved messages before they are provided to the QA agent.

Use the official evaluation procedure to report overall accuracy and category-level accuracy across the six question types: multi-hop reasoning, knowledge update, temporal reasoning, user-implicit reasoning, term ambiguity, and abstention.

Students are recommended to begin with the Finance and Technology domains, which provide solvability-filtered question sets, before optionally expanding to other domains.

Part 2: Analyze speaker- and group-related memory failures (40%)

Analyze failed examples from the baseline systems and investigate whether the failures are associated with insufficient use of multi-party conversational structure.

The analysis may include:

Speaker confusion: information from one participant is incorrectly attributed to another.

Group or channel confusion: information from one conversation group is incorrectly applied to another group.

Incorrect belief attribution: a statement made by one participant is treated as another participant's belief or preference.

Loss of reply context: a message is retrieved without correctly identifying which previous message it responds to.

Temporal update errors: outdated information is used after a participant has updated or corrected it.

Cross-participant reasoning failures: information from multiple participants is not correctly connected during reasoning.

Students should construct a failure taxonomy, report the distribution of different failure types, and provide representative examples. They should also investigate whether speaker, group, timestamp, and reply information could help recover the correct supporting memories.

Part 3: Speaker- and group-aware memory modeling (30%)

Based on the failure analysis, design and implement one focused modification to the baseline memory representation or retrieval mechanism.

A possible structured memory record contains five fields:

Memory = (speaker, group, time, reply-to, content)

Possible directions include:

speaker-aware retrieval or reranking;

group-aware memory retrieval;

metadata-aware indexing or filtering;

structured retrieval using speaker and group relations;

a lightweight conversational graph connecting speakers, messages, replies, and groups.

The proposed method should use conversational metadata during memory selection or ranking, rather than merely attaching metadata after content-based retrieval.

Compare the proposed method with the original BM25 and dense retrieval baselines under the same backbone LLM, embedding model, retrieval budget, and evaluation setting.

Report overall accuracy and category-level accuracy. Students should also analyze whether the proposed method retrieves more appropriate memories and reduces speaker- or group-related errors.

Use separate development and evaluation examples, and do not use reference answers or evaluation labels during retrieval.

The final submission should include runnable code, experimental results, representative failure cases, and a report of approximately 6 pages covering the methodology, experiments, and conclusions. A carefully analyzed negative result is acceptable. Recommended sections include an abstract, related work, methods, experimental results, and conclusions.

Part 4: Final report and demonstration — 10%

Submit a report of approximately 6 pages covering motivation, reproduction, error analysis, the proposed repair method, experiments, failure cases, and conclusions, together with runnable code and representative trajectories. Recommended sections include an abstract, related work, methods, experimental results, and conclusions.

Optional Models

Students may choose alternative open-source models, provided that all models can be downloaded and deployed locally on the available GPUs without relying on paid APIs.

- LLMs: Qwen3 series (including Qwen3.5 and other variants) with 4B–8B parameters, or LLaMA-3 series (including LLaMA-3.1, LLaMA-3.2, and other variants) with 3B–8B parameters.
- Text Embedding Models: E5 series (e.g., E5-small-v2, E5-base-v2, E5-large-v2) or other comparable open-source embedding models.

Students should use the same backbone LLM and embedding model across all baseline and proposed methods to ensure fair comparisons.

Dataset and code

GroupMemBench

GitHub: https://github.com/UCSB-NLP-Chang/GroupMemBench

GroupMemBench focuses on memory in multi-party conversations and provides synthetic multi-channel conversations, six question categories, BM25 and dense retrieval baselines, and evaluation scripts.

EverMemBench

GitHub: https://github.com/EverMind-AI/EverMemBench

EverMemBench evaluates long-horizon multi-party and multi-group conversational memory. It can be used as an additional benchmark to investigate whether the proposed memory mechanism generalizes to more complex collaborative dialogues.

Related work

[1] GroupMemBench: Benchmarking LLM Agent Memory in Multi-Party Conversations. NeurIPS 2026 (E&D).

[2] Evaluating Long-Horizon Memory for Multi-Party Collaborative Dialogues. KDD 2026.

GPU hours

< 50GPU hours

Project 13
Modeling Emotional Trajectories with Long-Term Memory for Personalized LLM Agents

Project manager: Jindong Li Email: jli839@connect.hkust-gz.edu.cn

Project description

Emotional-support agents often generate responses mainly from the user's current utterance or a short dialogue context. However, a user's emotional state may evolve across multiple sessions and may be influenced by earlier events, repeated concerns, interpersonal relationships, and unresolved experiences. Consequently, a single-turn emotion label or isolated episodic memory may be insufficient to capture the user's evolving long-term state.

This project studies whether explicitly modeling emotional trajectories in long-term memory can improve personalized emotional-support agents. Students will use ES-MemEval and its accompanying EvoEmo multi-session emotional-support dataset. The project focuses on connecting important historical events with evolving emotional states and investigating whether structured emotional memory provides additional information beyond conventional semantic retrieval.

The central research question is whether explicitly modeling the evolution of a user's emotional state can improve long-term user understanding and personalized response generation compared with conventional episodic memory or RAG.

This project focuses on conversational memory and evidence-grounded emotional-state modeling. It does not require clinical or psychiatric diagnosis.

Tasks

Part 1: Reproduce long-term emotional-memory baselines (20%)

Use ES-MemEval and EvoEmo to compare two basic settings:

Current Context Only: the model receives the current session context without explicit long-term memory.

Episodic RAG: semantically relevant historical conversations are retrieved and provided to the same answer model.

Students may reproduce the RAG pipeline from the official repository and implement the Current Context Only condition as an additional comparison baseline.

Use the ES-MemEval question-answering task as the primary quantitative evaluation. Report the official F1, BERTScore, and LLM-as-a-Judge metrics where applicable.

Pay particular attention to three memory capabilities:

Temporal Reasoning (TR): understanding events and user information across different times.

Conflict Detection (CD): identifying changes or conflicts in historical user information.

User Modeling (UM): understanding user characteristics, preferences, and needs from historical interactions.

Students should inspect several multi-session examples and explain why the current utterance or isolated historical snippets may be insufficient to understand the user's current situation.

Part 2: Analyze emotional-memory failures (40%)

Analyze failed examples from the episodic RAG baseline and investigate why conventional semantic retrieval may fail to capture the user's evolving emotional state.

Possible failure patterns include:

Missing historical events: important earlier experiences are not retrieved.

Irrelevant retrieval: recent but less important historical information is retrieved.

Transient-state confusion: a temporary emotional reaction is treated as a persistent emotional state.

Long-term pattern omission: recurring emotional patterns across sessions are ignored.

Missing event-emotion connections: the model fails to connect an earlier event with a later emotional response.

State-update failures: changes in the user's emotional state are not correctly reflected.

Overgeneralization: a stable user characteristic is inferred from insufficient conversational evidence.

Students should construct a failure taxonomy and analyze representative multi-session users.

The analysis should distinguish three levels of emotional information:

Immediate emotional reactions: emotions expressed in the current interaction.

Stage-level emotional states: emotions or concerns that persist across several related interactions.

Longer-term emotional tendencies: recurring patterns supported by multiple historical observations.

Students are encouraged to visualize how significant events, emotional reactions, and support needs evolve across sessions, while distinguishing observed dialogue evidence from inferred emotional states.

Part 3: Trajectory-aware emotional memory (30%)

Design and implement one lightweight emotional-memory representation that explicitly captures temporal changes in the user's emotional history.

A possible emotional memory record contains five fields:

Emotional Memory = (event, emotion, trigger, time, user need)

These fields are not assumed to be gold annotations provided by EvoEmo. Students may extract them from the historical dialogue using a fixed LLM prompt or another lightweight method.

After each session, the system extracts or updates a compact emotional-memory state based on the available conversational evidence.

A possible inference pipeline is:

Historical Conversations → Emotional Memory Extraction → Emotional Trajectory Update → Personalized Response

The final answer model may receive:

Current Context + Relevant Episodic Memories + Emotional Trajectory

Compare the following conditions:

Condition A: Current Context Only.

Condition B: Episodic RAG.

Condition C: Trajectory-Aware Emotional Memory.

Keep the backbone LLM, evaluation examples, and relevant retrieval settings unchanged. Students should use a consistent memory or context budget and report additional computational cost where applicable.

Use the ES-MemEval QA task as the primary quantitative evaluation, focusing particularly on Temporal Reasoning, Conflict Detection, and User Modeling. Report F1, BERTScore, and LLM-as-a-Judge scores using the official evaluation procedure.

Dialogue-generation examples may additionally be used for qualitative analysis of personalization, emotional appropriateness, and consistency with historical user information. These qualitative observations should be distinguished from the official QA evaluation results.

Students should also investigate regressions, including outdated emotional states, excessive reliance on old events, incorrect emotional aggregation, or unsupported assumptions about the user.

Emotional-memory extraction and updates must use only conversations that occurred before the evaluated interaction. Future sessions, reference answers, and evaluation annotations must not be used as memory during inference.

The final submission should include runnable code, experimental comparisons, representative emotional-trajectory cases, and a report of approximately 6 pages covering the methodology, results, and conclusions. Recommended sections include an abstract, related work, methods, experimental results, and conclusions.

Part 4: Final report and demonstration — 10%

Submit a report of approximately 6 pages covering motivation, reproduction, error analysis, the proposed repair method, experiments, failure cases, and conclusions, together with runnable code and representative trajectories. Recommended sections include an abstract, related work, methods, experimental results, and conclusions.

Optional Models

Students may choose alternative open-source models, provided that all models can be downloaded and deployed locally on the available GPUs without relying on paid APIs.

- LLMs: Qwen3 series (including Qwen3.5 and other variants) with 4B–8B parameters, or LLaMA-3 series (including LLaMA-3.1, LLaMA-3.2, and other variants) with 3B–8B parameters.
- Text Embedding Models: E5 series (e.g., E5-small-v2, E5-base-v2, E5-large-v2) or other comparable open-source embedding models.

Students should use the same backbone LLM and embedding model across all baseline and proposed methods to ensure fair comparisons.

Dataset and code

ES-MemEval / EvoEmo

GitHub: https://github.com/slptongji/ES-MemEval

ES-MemEval is the benchmark and evaluation framework, while EvoEmo is its accompanying multi-session emotional-support dataset.

The official repository provides the EvoEmo data in data/evo_emo.json, together with implementations for question answering, retrieval-augmented generation, summarization, dialogue generation, and evaluation.

Students are recommended to focus on the QA task for the main experiments and use selected dialogue-generation examples for supplementary analysis.

Related work

[1] ES-MemEval: Benchmarking Conversational Agents on Personalized Long-Term Emotional Support. WWW 2026.

[2] From Empathy to Personalized Empathy: Adapting Empathetic Strategies to Individual Users. Findings of EMNLP 2026.

GPU hours

< 50GPU hours

Project 14

Execution Feedback and Interaction Efficiency in CodeAct Agents

Project manager: Ruihan liu

Email: rliu674@connect.hkust-gz.edu.cn

Project Description

This project investigates how the presentation of execution feedback affects tool use in Large Language Model (LLM) agents. Students will work with M³ToolEval tasks involving information decoding, trade calculations, travel planning, and web queries. The goal is to understand how an agent uses tool-returned information to make subsequent decisions.

Students will build on the open-source CodeAct implementation. CodeAct represents actions as executable Python code, allowing agents to combine tools, preserve intermediate results, and adjust subsequent actions based on execution outputs or errors. Tool results may be stored in Python variables for later use or presented to the model as execution feedback. Different feedback strategies may affect how the model interprets tool results and how many interactions are required to complete a task.

The central research question is:

Can changing how execution feedback is selected and organized help an agent use tool results correctly while reducing interaction cost?

Students will reproduce the basic CodeAct workflow, implement one focused modification to execution feedback, and evaluate its effect through controlled experiments. The project does not require model training, fine-tuning, or the development of a new agent platform.

Codebases and Datasets

Baseline implementation: Official CodeAct repository.

https://github.com/xingyaoww/code-act

Evaluation environment: M³ToolEval from the official repository, using a designated course task subset.

https://github.com/xingyaoww/code-act/tree/d607f56c9cfe9e8632ebaf65dcaf2b4b7fe1c6f8/scripts/eval/m3tooleval

Base model: Qwen3.5-4B with local inference.

Course-provided materials: Fixed task IDs and development/evaluation split, to be released before the experiments begin.

Students should obtain the tasks and tools from the official repository and complete any necessary local environment configuration and adaptation. Developing new tools or a new evaluation environment is not required.

Part 1: Basic Reproduction (40%)

This phase familiarizes students with the CodeAct execution environment, tool-calling mechanism, and interaction workflow.

Task 1: Environment Familiarization (10%)

Run CodeAct in code-action mode and complete a short tool interaction.

Identify the task instruction, generated code action, tool return, model-visible feedback, and final answer. Explain how Python code invokes tools and how execution results become available in the next model input.

Verify whether Python variables persist across interaction turns within the same task, and check that the interpreter and tool state are correctly reset when a new task begins.

Expected output: A short interaction record demonstrating that the environment runs correctly, accompanied by a brief explanation of the task interface, action format, and execution workflow.

Students should use official tasks and tools rather than modify task definitions or the underlying behavior of the tools.

Task 2: Baseline Reproduction (20%)

Run the following two baseline conditions on the development tasks.

Condition A: Official Output. Preserve the original execution-output handling of the official executor.

Condition B: Full Trace. Preserve Python execution output while additionally exposing the names, arguments, return values, or exceptions of actual tool calls.

Save complete interaction trajectories, including task instructions, generated code actions, actual execution results, model-visible feedback, and final answers. Demonstrate successful trajectories and document failures observed during the experiments.

Expected output: Baseline results for Conditions A and B on the development set, together with complete interaction records illustrating agent behavior under both conditions.

Students are expected to reproduce the CodeAct execution and evaluation workflow, not the exact success rates reported in the original paper.

Task 3: Execution State and Feedback Inspection (10%)

Trace one interaction containing multiple tool calls and examine how tool results are stored, displayed, and used.

Compare the actual tool returns, the program state stored in Python variables, and the feedback received by the model in the next turn. Check whether subsequent actions correctly use the available information.

For example, an intermediate tool result may not be displayed directly to the model but may already be stored in a Python variable and correctly used by later code. Therefore, feedback sufficiency should be assessed using both program state and subsequent actions, rather than model-visible text alone.

Expected output: An annotated interaction example showing the relationships among tool returns, program state, model-visible feedback, and subsequent actions.

Part 2: Feedback Design and Evaluation (40%)

This phase focuses on one manageable modification to execution feedback and evaluates it through controlled comparisons.

Task 4: Student-Designed Feedback Strategy (20%)

Implement Condition C by modifying how execution information is selected and organized before being returned to the model.

One suggested direction is to selectively expose tool results that are not already visible in the execution output. The design may consider what information is already included in the existing execution output, which previously hidden tool results should be added to the model-visible feedback, and how the relationships among tool calls, arguments, and return values can be preserved.

Students may propose other reasonable designs, but the modification should focus on one primary feedback strategy. They should specify what information the strategy uses, when it is triggered, and how it is expected to affect subsequent agent behavior.

Since Condition B already exposes complete tool feedback, Condition C should introduce a concrete information-selection rule rather than simply displaying all tool results again.

The model, action-generation prompt, existing examples, and tool implementations must remain unchanged. Develop and revise the feedback strategy using development tasks only. Feedback decisions must depend solely on information already obtained during the current task, without access to reference answers or other evaluation information.

Expected output: The final feedback strategy, a brief algorithmic description, and representative examples collected during development.

Task 5: Controlled Comparison (20%)

Evaluate Conditions A, B, and C using the shared experimental protocol below, and include at least one ablation condition, Condition D.

The ablation should remove one core rule from Condition C while keeping other settings unchanged. For example, students may retain the same feedback format but disable the rule that supplements hidden tool results. Verify whether the rule is actually triggered and whether removing it changes model-visible feedback or subsequent agent behavior.

Compare task success, interaction counts, and model-use cost under the same tasks, initial states, and execution budgets.

Discuss whether the observed results support the expected benefits of the proposed strategy. Include tasks where the modification provides no improvement or causes performance degradation. If a rule is never triggered, its effect should be reported as unverified. If an ablation is identical to an existing baseline, the corresponding results may be reused.

Expected output: A complete results table covering all experimental conditions, an ablation comparison, and an evidence-based analysis supported by interaction records.

The feedback modification is not assumed to improve task success or reduce interaction cost. A carefully implemented and evaluated negative result is acceptable.

Part 3: Error Analysis and Presentation (20%)

This phase examines whether execution feedback changes actual agent behavior and communicates the experimental findings.

Task 6: Feedback and Behavioral Analysis (10%)

Where sufficient data are available, analyze 5–10 tasks using paired interaction trajectories from different feedback conditions.

For each case, compare the actual tool returns, model-visible feedback, and subsequent actions. Distinguish among the following situations.

Insufficient feedback: Information needed for a subsequent decision was not presented to the model.

Incorrect information use: The required information was visible, but the model misinterpreted or failed to use it correctly.

Behavioral change: The modified feedback changed relevant subsequent actions, whether or not the task ultimately succeeded.

New problem introduced: The feedback strategy caused information interference, incorrect associations, or unnecessary interactions.

Also inspect other failure sources, such as code errors, invalid tool arguments, and execution-budget limits. Avoid attributing all failures to the feedback strategy.

Include contrasting outcomes when observed. Do not select only successful examples or invent cases to fill predefined categories. All conclusions should be supported by execution evidence.

Expected output: A case-analysis table with short excerpts from interaction records supporting the assigned behavioral categories.

Task 7: Final Report and Demonstration (10%)

Submit a reproducible project package, a concise research report, and a short demonstration.

Clearly identify which existing components were reused and which were modified. Summarize whether the proposed feedback strategy was effective under the tested conditions, discuss its limitations, and propose one small future experiment.

The demonstration should show the same task running under different feedback conditions, highlighting changes in model-visible feedback and subsequent actions.

Expected output: The materials specified in the Final Report and Deliverables section below.

Grading emphasizes correct implementation, fair comparison, and evidence-based analysis. A well-executed project can receive full marks even when the student-designed strategy does not outperform the baselines.

Shared Experimental Protocol

Task Split and Strategy Selection

Use a fixed subset of 43 M³ToolEval tasks, consisting of 16 development tasks and 27 evaluation tasks. The specific task IDs and split will be released before the experiments begin.

Use development tasks only for feedback-strategy design and parameter selection, comparing at most three candidate configurations. Freeze the final configuration before running the evaluation set.

Retain results for every evaluation task, including failed tasks and tasks without a submitted final answer. Evaluation examples must not be filtered or excluded based on experimental outcomes.

Model and Execution Settings

All experiments must use the same version of Qwen3.5-4B with local inference, BF16 precision, greedy decoding, and non-thinking mode.

All conditions must use identical prompts, action parsing, tool implementations, initial task states, and resource limits. Specific interaction and execution limits will be standardized before the experiments begin.

A task ends when the agent first submits a final answer. Reference answers must be used for offline scoring only and must not be provided to the agent or the feedback strategy during execution.

Evaluation Metrics

Report overall task success rate and success rates by task category to measure task performance.

Report the number of model calls and tool calls to measure interaction cost.

Report input and output token counts to measure model-use cost.

Report total task wall-clock time and peak GPU memory usage to measure execution cost.

Cost accounting should include feedback-processing overhead and cover all tasks, including failures and tasks without final answers.

Retain per-task results and compare feedback conditions using both observed behavioral changes and measured costs.

Conclusions should be limited to the fixed task subset evaluated in this project and should not be generalized directly to the full M³ToolEval benchmark or other agent environments.

Final Report and Deliverables

Submit a research report of **up to six pages**, excluding references and appendices.

The report should cover research motivation, methods, experimental setup, results, ablation studies, error analysis, conclusions, and limitations.

The submission must also include runnable code, dependency specifications, version records, experiment configurations, task IDs, raw predictions, and per-task evaluation results.

Students must preserve complete interaction logs, including code actions, execution results, model-visible feedback, and final answers. GPU usage records and individual contribution statements must also be submitted.

A poster should be submitted according to the course-wide requirements.

Related Work and Open-Source Resources

[1] Wang et al. Executable Code Actions Elicit Better LLM Agents. ICML 2024.

Paper: https://arxiv.org/abs/2402.01030

Official code: https://github.com/xingyaoww/code-act

GPU Requirement

< 100 GPU hours.

Project 15

Understanding and Mitigating Overthinking in Large Reasoning Models

Project manager: Ruochen Liu

Email: rliu190@connect.hkust-gz.edu.cn

Project Description

Large reasoning models (LRMs) achieve strong performance through extended reasoning, but often exhibit overthinking, generating unnecessarily long reasoning processes without improving answer accuracy. Reducing redundant reasoning while preserving correctness is important for efficient inference.

This project investigates two approaches. **Reasoning Strength Steering** controls reasoning length by modifying internal model representations, while **Stepwise Adaptive Thinking (SAT)** dynamically adjusts reasoning strategies using a lightweight process reward model and a finite-state machine. These methods provide different perspectives on controlling reasoning effort.

Students will reproduce both methods, investigate their strengths and limitations, study the tradeoff between reasoning accuracy and efficiency, and explore a simple improvement to one of the methods.

References:

- On Reasoning Strength Planning in Large Reasoning Models (NeurIPS 2025)
- SAT: Balancing Reasoning Accuracy and Efficiency with Stepwise Adaptive Thinking (ACL 2026)

Objectives

- Reproduce two methods for mitigating overthinking in reasoning models.
- Understand the differences between activation steering and step-level adaptive reasoning control.
- Investigate their effectiveness and limitations in balancing reasoning accuracy and efficiency.
- Implement and evaluate one modest improvement to an existing method.

Codebases and Datasets

- **Reasoning Strength Steering:** https://github.com/AlphaLab-USTC/LRM-plans-CoT
- **Stepwise Adaptive Thinking:** https://github.com/byxw13/SAT_Code
- **Model:** DeepSeek-R1-Distill-Qwen-1.5B (recommended).
- **Dataset:** GSM8K, using approximately 100–200 test questions. MATH-500 is optional.

One inference model and one dataset are sufficient. Students may use the pretrained PRM provided by SAT. No additional model comparisons or large-scale training are required.

Tasks

Part 1: Reproduce the Basic Methods (30%)

- Run the Steering pipeline: extract reasoning-strength vectors, apply activation interventions, and evaluate their effects on reasoning length and accuracy.
- Run the SAT pipeline: assess reasoning steps using PRM scores, apply finite-state control, and dynamically adjust reasoning strategies.

● Reproduce one representative experimental result from each repository on a manageable subset.

● Explain the input, output, control mechanism, and main assumptions of each method.

Report answer accuracy, average reasoning length, token reduction, and inference time for both methods and the original model.

Part 2: Understand Suitability and Failure Cases (25%)

Use experimental examples to investigate the following questions:

● Under what circumstances does each method effectively reduce overthinking while maintaining accuracy?

● What are the typical failure cases of Steering and SAT, and what causes them?

● Does shorter reasoning necessarily mean that redundant reasoning has been removed?

Present several successful and unsuccessful examples. Explain the differences between the two approaches and their respective limitations.

Part 3: Conduct a Controlled Empirical Study (20%)

● Select **one method** and investigate one factor affecting its performance, such as steering strength, intervention layer, or SAT control threshold.

● Evaluate several configurations and visualize the tradeoff between answer accuracy and reasoning-token usage.

● Explain the observed trends and unexpected results.

● Keep test examples separate from parameter tuning and steering-vector construction. Reuse generated outputs where possible.

Reproducing every experiment from the original papers is unnecessary.

Part 4: Explore a Simple Methodological Improvement (25%)

Select one of the two methods and implement a small improvement motivated by its observed limitations.

Suggested directions include:

Option A: Improving Steering

Explore an adaptive steering strategy that adjusts intervention strength or timing according to question characteristics or reasoning signals, rather than applying a fixed intervention.

Option B: Improving SAT

Explore a more efficient or robust step-level controller, such as reducing unnecessary PRM evaluations or improving state-transition decisions.

Students should:

1. Identify a limitation and explain the motivation for the proposed improvement.
2. Implement the improvement and compare it with the original method.

3. Conduct a small ablation study and analyze representative failure cases.

Alternative well-motivated improvements are welcome. A well-executed negative result is acceptable. Assessment should reward sound reasoning, reproducible experiments, and honest analysis rather than performance improvement alone.

Final Report and Deliverables

Submit a report covering motivation, methods, experimental setup, reproduction results, failure analysis, controlled experiments, proposed improvement, and conclusions, together with runnable code, configurations, plots, a poster, and individual contribution statements.

The task percentages above apply to the submitted project package; the poster can retain the separate weighting in the course template.

Recommended Computational Scope

< 100 GPU hours